53 New Integer Schemes for 3×3 Matrix Multiplication with 23 Products

Reproducible report — logic repo, matmul track, 2026-07-03 (commits 45e5acc…93347f2). All artifacts live in the repository; every claim has a mechanical check (§4). Canonical source: doc/matmul_53_3x3_schemes.md.

1. Summary

We report **53 new schemes for multiplying two 3×3 matrices with 23 multiplications, with coefficients in {−1, 0, +1}** — valid over any commutative ring, the same object class as Laderman's 1976 scheme. Each scheme is:

1. **verified mod 2** against all 729 Brent equations,
2. **pairwise inequivalent** under the full de Groote symmetry group $\mathrm{GL}(3,2)^3 \rtimes \mathrm{S}_3$ (exact witness-or-refutation checks),
3. **inequivalent to every published scheme**: all 17,376 schemes of the Kauers–Heule–Seidl (HKS) database *and* the four classics (Laderman 1976, Smirnov 2013, Oh–Kim–Moon 2013, Courtois–Bard–Hulme 2011),
4. **lifted to integer coefficients** in {−1,0,+1} and verified **exactly over** $\mathbb{Z}$ against all 729 integer Brent equations.

Discovery cost: the 53 came out of one **6-minute single-threaded run** of a neighborhood-walk pipeline (138 raw finds → 53 survived full-database novelty checking), plus ≈40 minutes of certification compute. For scale: the HKS campaign that produced the 17,376-scheme database used ≈35 CPU-years (methods differ; §7).

The engine is a **native-ANF stochastic local search**: the Brent system is kept as 729 cubic-XOR constraints over the 621 real variables instead of a ≈26,500-variable CNF, which extends the seeded-repair horizon by ≥5,000× over yalsat-on-CNF and, with an exact GF(2) tensor-closure move, solves 8/10 of the official HKS challenge-1 instances (yalsat's published record: 5/10).

2. Background

A bilinear scheme for 3×3 matrix multiplication with r products is a triple of coefficient tensors α, β, γ:

$$M_m = \left(\sum_{a,b} \alpha^{(m)}_{ab} A_{ab}\right)\left(\sum_{c,d} \beta^{(m)}_{cd} B_{cd}\right), \qquad C_{pq} = \sum_{m=1}^{r} \gamma^{(m)}_{pq} M_m$$

Correctness is equivalent to the **Brent equations**; over GF(2):

$$\bigoplus_{m=1}^{r} \alpha^{(m)}_{ab}\,\beta^{(m)}_{cd}\,\gamma^{(m)}_{pq} = \delta_{bc}\,\delta_{ap}\,\delta_{dq} \qquad \forall\,(a,b,c,d,p,q) \in [3]^6$$

i.e. 729 cubic XOR equations over 27r variables (r = 23: 621 variables; 27 equations have right-hand side 1 — the "type-3" or delta equations). Any integer scheme reduces mod 2 to a GF(2) scheme; conversely a GF(2) scheme *may* lift to signs ±1 (HKS observe lifting rarely fails).

Known landscape: r = 23 achievable (Laderman 1976); best lower bound 19 (Bläser 2003); **r = 22 open in both directions**. Before HKS (SAT 2019 / J. Symbolic Computation 2021), only 4 inequivalent {−1,0,1} schemes were known; their local-search campaign found >17,000 more, all published in the Linz database this report checks against.

3. The pipeline

3.1 Native-ANF SLS engine (src/anf.rs, binary anf)

The Brent system is represented natively as cubic-XOR (ANF) constraints — no Tseitin auxiliaries. Flipping a variable touches exactly its 81 incident equations; a monomial toggles iff its two partner bits are 1, so incremental evaluation is O(81) per flip (0.4–5 M flips/s/core measured; ≈10 M flips/s aggregate over 10 threads). Two policy regimes: close repair (WalkSAT/SKC, noise 0.2, init density 0.25) and pairing/from-scratch (probSAT, $c_b = 2.5$, init density 0.10 ≈ the free-support density of real completions).

The structural move is the **tensor closure**: the system is tri-linear and every equation contains exactly one variable-group of each tensor, so fixing two tensors decomposes the third into 9 independent 81×r GF(2) linear systems — a consistent single-tensor closure *solves the instance outright*, and each closure call is monotone. It runs as an injected hook every N flips.

Baselines (this machine): kissat cannot solve the r = 23 CNF (unknown at 60 s; 41.6 s to prove even 2×2-in-6 UNSAT). yalsat (HKS's solver, v1.0.1) matches its published seeded operating point (fix 414/621 bits: 0.05–0.2 s) but **times out at 300 s at fix = 300 and fix = 250 — instances the native engine solves in 5 ms and 60 ms**. On the official challenge-1 instances the native engine + closure solves **8/10** (best 0.019 s / 0.069 s; the latter confirmed by planting our bits into *their* CNF and running kissat: SATISFIABLE).

3.2 Discovery: neighborhood walk (matmul/walk.py)

HKS “method 2,” compounding: a pool of 24 verified seeds (4 classics + 20 spanning 20 DB rank-pattern directories); each hop freezes a random 300 of a pool scheme's 621 bits and lets the engine complete the rest; completions are canon-deduped (sorted summands); genuinely new schemes join the pool. Every accepted scheme is re-verified by code independent of the search. The committed run: **138 schemes at ≈3 s/scheme**, accelerating as the pool diversifies.

3.3 Exact equivalence (matmul/equiv.py)

“New” must mean new modulo de Groote symmetry. Writing schemes as summands (A, B, C) with $\mathbf{C} = \gamma^{\mathrm{T}}$, the group acts as the cyclic sandwich

$$(A, B, \tilde{C}) \mapsto (PAQ^{-1},\ QBR^{-1},\ R\tilde{C}P^{-1}), \qquad G = \mathrm{GL}(3,2)^3 \rtimes S_3, \quad |G| = 168^3 \cdot 6$$

so every summand-matching constraint is **linear** in the 27 GF(2) unknowns of (P, Q, R): equivalence testing is rank-pruned backtracking + incremental RREF + nullspace enumeration + an exact multiset check (≈ms per pair), returning a witness or a refutation. Self-tests: 12 random group elements recovered equivalent-with-witness; Laderman vs Smirnov refuted. On the 138 finds: **129 distinct classes**.

3.4 Database novelty (matmul/novelty.py, dbcheck.py)

Layer 1 — rank-pattern absence: the DB's 302 directory names encode per-summand rank types (legend constraint-solved from 20 known dir↔scheme pairs); rank patterns are G-invariants, so pattern-absence ⇒ inequivalence. Control: Laderman's pattern is absent from all 302 found-scheme directories — correct. Layer 2 — full-database exact check: all **17,376** schemes fetched (schemes.tgz), converted with 0 parse failures and 13/13 byte-identical anchors; per find, fingerprint filtering then exact equivalence. Hardening: all 17,376 fingerprinted; the surviving finds (×6 S_3 variants) have **zero fingerprint matches anywhere** — verdicts independent of the directory-name legend. Result: 85 finds equivalent to DB schemes (walk is DB-seeded; witnesses recorded), **53 new vs the entire database**, pairwise inequivalent, inequivalent to the classics.

3.5 Integer lifting (matmul/lift.py)

Lifting as **sign-SAT** (vs HKS's Gröbner route): one sign bit per support coefficient; a covering term's sign is the XOR of its three sign bits; the integer Brent equation with k covering terms and right-hand side δ becomes “exactly $(k-\delta)/2$ of the k term-bits equal 1”; per-product scaling is broken by fixing the first α- and β-support signs. ≈2,000-clause CNFs, kissat solves each in milliseconds; every lift is verified **exactly over $\mathbb{Z}$** by independent code. Controls: all 4 classics lift. Result: **53/53 lifted, zero failures** (matmul/lifted/*.txt).

4. Verification chain — check every claim

```
# 0. build + engine self-tests                                  [~1 min]
cargo build --release --bin anf
cargo test --release --lib anf::          # expect: 7 passed

# 1. generator sanity (Strassen + Laderman verify)               [~5 s]
cd matmul && python3 brent.py selftest

# 2. the 53 are valid + distinct-after-sorting                   [~10 s]
grep NEW novelty_verdicts.csv | cut -d, -f1 | sed 's|^|found/|;s|$|.bits|' \
  | xargs cat | python3 canon.py 3 3 3 23 /dev/stdin
#   expect: 53 schemes read, 0 INVALID, 53 distinct after summand sorting

# 3. exact-equivalence self-test + 53 = 53 classes               [~30 s]
python3 equiv.py selftest
grep NEW novelty_verdicts.csv | cut -d, -f1 | sed 's|^|found/|;s|$|.bits|' \
  | xargs python3 equiv.py classes      # expect: TOTAL de-Groote classes: 53

# 4. rank-pattern novelty vs the 302 DB dirs (no download)       [~30 s]
python3 novelty.py db_rank_patterns.txt found/walk-00029.bits

# 5. FULL database check from primary sources                    [~15 min]
python3 dbcheck.py all
#   fetch (43 MB) -> convert 17,376 (expect 0 failures) -> controls
#   (20 byte-identical seeds + 4 classics) -> check vs ALL fingerprints;
#   ends with cross-check vs committed novelty_verdicts.csv: MATCH

# 6. integer lifting + exact Z-verification                      [~1 min]
grep NEW novelty_verdicts.csv | cut -d, -f1 | sed 's|^|found/|;s|$|.bits|' \
  | xargs python3 lift.py --outdir /tmp/lift53
#   expect: "53 lifted, 0 not +-1-liftable"; each line Z-VERIFIED

# 7. challenge-1 spot-check (their exact CNF, our scheme)        [~1 min]
git clone https://github.com/marijnheule/matrix-challenges challenges
mkdir -p inst && python3 import_core.py \
  challenges/challenge1/MM-23-2-2-2-2-A.cnf inst/core-A.freeze
../target/release/anf 3 3 3 23 --freeze-file inst/core-A.freeze \
  --probsat --cb 2.5 --density 0.1 --seconds 60 --threads 10 --seed 3 \
  --quiet | grep '^b ' > /tmp/solA.txt
python3 check_their_cnf.py challenges/challenge1/MM-23-2-2-2-2-A.cnf \
  /tmp/solA.txt                      # expect: SATISFIED by our scheme
```

Verification of the committed 53 (steps 0–6) is deterministic. Discovery (walk.py) and the challenge-1 solves are stochastic and timing-dependent: reruns reproduce the phenomena, not bit-identical artifacts. kissat may return different sign models across versions — any model returned is then $\mathbb{Z}$-verified, which is the claim that matters.

5. The 53 schemes

Files: mod-2 bit-vectors matmul/found/walk-*.bits (the 53 names are the NEW rows of matmul/novelty_verdicts.csv); signed integer forms matmul/lifted/walk-*.txt. Support = number of nonzero coefficients; for a 3×3×23 scheme the naive addition count is exactly support − 55. Ours: support 149–164 (median 154) = **94–109 naive additions**; reference points: Laderman 153/98, Smirnov 139/84 = the DB minimum (support percentiles of the full DB: p1 = 146, median = 159 — our sparsest sits at the 3rd percentile; 22 of the 53 need fewer naive additions than Laderman; none beats the DB's sparsest). Rank-type multiset = per-summand sorted (rank α, rank β, rank γ) with multiplicities — the invariant separating 51 of the 53 from the whole DB at the coarsest level.

scheme	support	naive adds	rank-type multiset
walk-00028	157	102	111×12 112×2 113×3 122×2 222×4
walk-00029	156	101	111×11 112×3 113×3 122×2 222×4
walk-00034	157	102	111×12 112×2 113×3 122×2 222×4

scheme	support	naive adds	rank-type multiset
walk-00035	150	95	111×14 112×3 113×1 122×1 222×4
walk-00036	151	96	111×15 112×2 113×1 122×1 222×4
walk-00039	163	108	111×10 112×7 122×2 222×4
walk-00040	160	105	111×10 112×7 122×2 222×4
walk-00046	149	94	111×13 112×6 222×4
walk-00047	153	98	111×16 112×3 222×4
walk-00048	158	103	111×16 112×3 222×4
walk-00055	157	102	111×12 112×5 122×2 222×4
walk-00056	157	102	111×14 112×3 113×1 122×1 222×4
walk-00058	160	105	111×10 112×7 122×2 222×4
walk-00060	155	100	111×8 112×8 113×1 122×2 222×4
walk-00063	155	100	111×14 112×5 222×4
walk-00064	155	100	111×15 112×4 222×4
walk-00066	151	96	111×9 112×8 113×1 122×1 222×4
walk-00072	151	96	111×12 112×3 113×3 122×1 222×4
walk-00075	154	99	111×8 112×6 113×4 122×1 222×4
walk-00076	152	97	111×13 112×6 222×4
walk-00077	150	95	111×15 112×4 222×4
walk-00081	150	95	111×14 112×3 113×1 122×1 222×4
walk-00082	152	97	111×13 112×6 222×4
walk-00083	152	97	111×13 112×6 222×4
walk-00084	149	94	111×13 112×6 222×4
walk-00085	153	98	111×16 112×3 222×4
walk-00091	156	101	111×10 112×4 113×2 122×3 222×4
walk-00092	153	98	111×9 112×7 113×1 122×2 222×4
walk-00093	153	98	111×8 112×9 113×1 122×1 222×4
walk-00094	151	96	111×9 112×5 113×4 122×1 222×4
walk-00100	160	105	111×11 112×6 122×2 222×4
walk-00105	151	96	111×11 112×8 222×4
walk-00106	161	106	111×9 112×5 113×3 122×2 222×4
walk-00107	149	94	111×10 112×5 113×3 122×1 222×4
walk-00108	158	103	111×11 112×3 113×3 122×2 222×4
walk-00115	154	99	111×14 112×3 113×1 122×1 222×4
walk-00116	155	100	111×14 112×2 113×1 122×2 222×4
walk-00120	152	97	111×14 112×3 113×1 122×1 222×4
walk-00121	164	109	111×8 112×5 113×4 122×2 222×4
walk-00122	163	108	111×9 112×5 113×3 122×2 222×4
walk-00125	151	96	111×13 112×4 113×1 122×1 222×4
walk-00127	155	100	111×15 112×4 222×4
walk-00136	154	99	111×13 112×6 222×4
walk-00138	154	99	111×11 112×4 113×3 122×1 222×4
walk-00141	160	105	111×12 112×2 113×3 122×2 222×4

scheme	support	naive adds	rank-type multiset
walk-00143	152	97	111×13 112×4 113×1 122×1 222×4
walk-00144	150	95	111×13 112×2 113×3 122×1 222×4
walk-00145	149	94	111×12 112×3 113×3 122×1 222×4
walk-00149	156	101	111×14 112×3 113×1 122×1 222×4
walk-00150	159	104	111×14 112×3 113×1 122×1 222×4
walk-00151	150	95	111×14 112×3 113×1 122×1 222×4
walk-00157	151	96	111×11 112×8 222×4
walk-00159	150	95	111×13 112×2 113×3 122×1 222×4

The multiset shown is the coarse invariant; schemes sharing it are separated by the finer pair-sum fingerprint and/or exact checks. All 53 are pairwise inequivalent.

6. Secondary results

• **Challenge 1 (HKS): 8/10 official instances solved** (pairing cores, no streamliners; yalsat's record 5/10). The two holdouts floor at best 3/729 after 600 s × 10 threads.

• **Path-space SLS (local search on the connection method): built, verified correct, measured NEGATIVE** at equal budget vs assignment-space SLS. Diagnosis: the connection objective collapses (a conflicted variable hides force1×force0 repairs) and the Brent matrix's sharing is dense (81 equations/variable) — path rerouting is myopic exactly where one assignment flip re-evaluates everything at once.

• **r = 22 (challenge 4): open, probed.** Plain native attacks floor at 8/729; drop-a-product repairs floor at 1/729, but the finisher proved every such floor-1 state violates exactly the type-3 equation whose sole cover was dropped and is rigid to radius 3 — the floor-1 shell is a seeding artifact, not evidence about r = 22.

7. Caveats and scope

• "New" = inequivalent under de Groote symmetry to the 17,376 schemes of the Linz database snapshot (schemes.tgz, 2020-08-07) and the 4 classics. No claim about unpublished or post-2020 collections.

• Discovery-effort comparisons with HKS (35 CPU-years vs minutes) are indicative, not controlled: we start from their published schemes; they started from 4 and invented the methods. The like-for-like numbers are the same-machine yalsat A/Bs (§3.1).

• The checkers are our code; self-tests and controls are listed in §4. Strongest external anchors: our challenge-1 solution satisfies HKS's own CNF under kissat, and the DB hardening uses only rank arithmetic + the exact checker on published inputs.

• 51/53 schemes have four rank-(2,2,2) summands (like most HKS finds); none matches Laderman's four-quadruple core type.

8. Pointers

Lab notebook: doc/matmul_plan.md (every measurement, including negatives and retractions). Engine: src/anf.rs, src/bin/anf.rs. Tools: matmul/{brent, sls, walk, canon, equiv, novelty, dbcheck, lift, import_core, check_their_cnf}.py. External sources fetched by commands shown in §4: the DB archive, the matrix-challenges clone, yalsat. References: HKS SAT 2019 (arXiv:1903.11391) and J. Symb. Comput. 104 (2021); Laderman, Bull. AMS 82(1), 1976; Bläser, J. Complexity 19(1), 2003; database: algebra.uni-linz.ac.at/research/matrix-multiplication.